

Reproducing ResNet on CIFAR-10: Experiments on Network Depth, Batch Size, and Pooling

Haeyoon Shin

1. Main Implementation and Experiment

As the field of deep learning advances, prior research has highlighted the importance of deeper networks to build better models. However, as more layers are stacked in a model, it often confronts a limitation called "degradation." This issue arises not from overfitting but rather from the challenges in optimizing deeper networks. In the paper titled 'Deep Residual Learning for Image Recognition'[1], the authors present evidence of degradation problem by comparing the training and test errors of 20-layer and 56-layer plain networks. Both the training and test errors were higher in the 56-layer plain networks compared to the 20-layer plain networks. To overcome the degradation problem, the paper proposes a deep residual learning framework. Deep residual learning framework perform identity mapping by adding shortcut connections to the neural network. Experimental results across various datasets demonstrate that the deep residual learning framework consistently effectively achieves lower training and test errors as the network depth increases.

Based on the publicly available ResNet18 code[2], I constructed the ResNet20 code as outlined in the Deep Residual Learning for Image Recognition paper. Several adjustments were made to the ResNet18 code to align with the specifications detailed in the paper. The number of layers and filters were adjusted according to the specifications outlined in the research paper. To describe in more detail, the specific structure of ResNet20 matches a specified Table 1. While the publicly available ResNet18 code utilized shortcut connection option B, the ResNet20 model in the paper employs shortcut connection option A. As a result, shortcut connection option A is implemented in two different ways in the code. One method involved creating a separate `ShortcutLayer` class to generate the layer, which was then used into the `BasicBlock` class to add the shortcut connection. The other method involved defining a function named `_shortcut_layer` within the `BasicBlock` class to add the shortcut connection. The weight decay, momentum, and training batch size were set to 0.0001, 0.9, and 128, respectively, as specified in the paper. The initial learning rate was set to 0.1. It was then divided by 10 at 100 and 150 epochs, following the ratio of iterations at which the

learning rate decreases mentioned in the paper. The pooling code was modified to perform global pooling by using `'F.avg_pool2d(out, out.size()[3])'`. The data augmentation techniques used during model training were consistent with those described in the paper and implemented in the publicly available ResNet18 code.

When trained on the CIFAR-10 dataset, the ResNet20 model had 269,722 parameters. The training error was 0.41%, and the test error was 8.29%. These results closely resembled the reported test error of 8.75% in the paper for ResNet20.

2. Sub Experiments

2.1. Network Depth

To identify whether ResNet resolves the degradation problem, I trained not only ResNet20 model but also ResNet32, ResNet44, and ResNet56 models. The training errors decreased as the network depth increased, with ResNet20, ResNet32, ResNet44, and ResNet56 achieving training errors of 0.41%, 0.11%, 0.10%, and 0.05%, respectively (Table 2). These results strongly suggest that implementing identity mapping in ResNet effectively mitigates the degradation problem. However, the test errors for ResNet20, ResNet32, ResNet44, and ResNet56 were 8.29%, 7.11%, 7.96%, and 7.85%, respectively (Table 2). Under the conditions of this project, including batch size, learning rate, and number of epochs, ResNet32 showed the best performance. The result may be attributed to overfitting problem. Although the test error did not decrease as the network depth increased, the decrease in training error with increasing network depth suggests that the model may be overfitting to the training data.

2.2. Batch Size

In the paper, the batch size is set at 128, but I explore how the model's performance would change with different batch sizes. Therefore, I trained the model with batch sizes of 64, 128, and 256. When the model was trained with batch sizes of 64, 128, and 256, the test errors were 7.59%, 8.29%, and 8.83%, respectively (Table 3). Additionally, the time taken to train one epoch with batch sizes of 64, 128, and

layer name	output size	20-layer			32-layer			44-layer			56-layer		
conv1	32×32	$3 \times 3, 16$											
conv2_n	32×32	$\begin{bmatrix} 3 \times 3, 16 \\ 3 \times 3, 16 \end{bmatrix}$	$\times 3$	$\begin{bmatrix} 3 \times 3, 16 \\ 3 \times 3, 16 \end{bmatrix}$	$\times 5$	$\begin{bmatrix} 3 \times 3, 16 \\ 3 \times 3, 16 \end{bmatrix}$	$\times 7$	$\begin{bmatrix} 3 \times 3, 16 \\ 3 \times 3, 16 \end{bmatrix}$	$\times 9$				
conv3_n	16×16	$\begin{bmatrix} 3 \times 3, 32 \\ 3 \times 3, 32 \end{bmatrix}$	$\times 3$	$\begin{bmatrix} 3 \times 3, 32 \\ 3 \times 3, 32 \end{bmatrix}$	$\times 5$	$\begin{bmatrix} 3 \times 3, 32 \\ 3 \times 3, 32 \end{bmatrix}$	$\times 7$	$\begin{bmatrix} 3 \times 3, 32 \\ 3 \times 3, 32 \end{bmatrix}$	$\times 9$				
conv4_n	8×8	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix}$	$\times 3$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix}$	$\times 5$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix}$	$\times 7$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix}$	$\times 9$				
	1×1	average pool, softmax											

Table 1. Architecture for ResNet20, ResNet32, ResNet44, ResNet56

method	# layers	# params	error (%)	
			training	test
ResNet	20	269,722	0.41	8.29
ResNet	32	464,154	0.11	7.11
ResNet	44	658,586	0.10	7.96
ResNet	56	853,018	0.05	7.85

Table 2. Training and test error (%) of ResNet20, ResNet32, ResNet44, ResNet56 on the CIFAR-10 data set

method	batch size	test error (%)	time (sec)
ResNet20	64	7.59	31.5
ResNet20	128	8.29	23.6
ResNet20	256	8.83	9.4

Table 3. Test error (%) and training time (sec/epoch) of the model depending on batch size

256 was 31.5 seconds, 23.6 seconds, and 9.4 seconds, respectively (Table 3). Increasing the batch size reduces the number of training steps per epoch, leading to a decrease in overall training time. However, as indicated by the increase in test error with larger batch sizes, a larger batch size does not necessarily lead to more effective optimization. Larger batch sizes may result in difficulty escaping from local minima. Therefore, batch size significantly influences both training speed and model performance, and selecting the most suitable batch size for the model through experimentation is critical.

2.3. Pooling

As discussed in class, I examined how replacing average pooling with convolution affects the model's performance. Because using convolution instead of average pooling has the same mathematical structure, the code runs without errors. However, when using convolution instead of average pooling, the training loss becomes NaN and does not con-

method	pooling code	# params	loss
ResNet20	F.avg_pool2d(out,out.size()[3])	269,722	0.016 (train) 0.377 (test)
ResNet20	nn.Conv2d(64,64, kernel_size=8, stride=1, padding=0, bias=False)	531,866	NaN (train) NaN (test)

Table 4. Number of parameters and loss values for the average pooling method and convolution method

verge (Table 4). While average pooling calculates the average value for all elements of the input, preserving all features of the image, convolution preserves only partial patterns of the input, leading to information loss. Moreover, using convolution instead of average pooling requires more parameter calculation (Table 4). Therefore, although the methods of reducing input size using average pooling and convolution may be mathematically equivalent, it is preferable to minimize information loss by using average pooling.

References

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 1
- [2] Kuang Liu. Pytorch cifar. GitHub, 2021. Accessed: 2024-05-01. 1